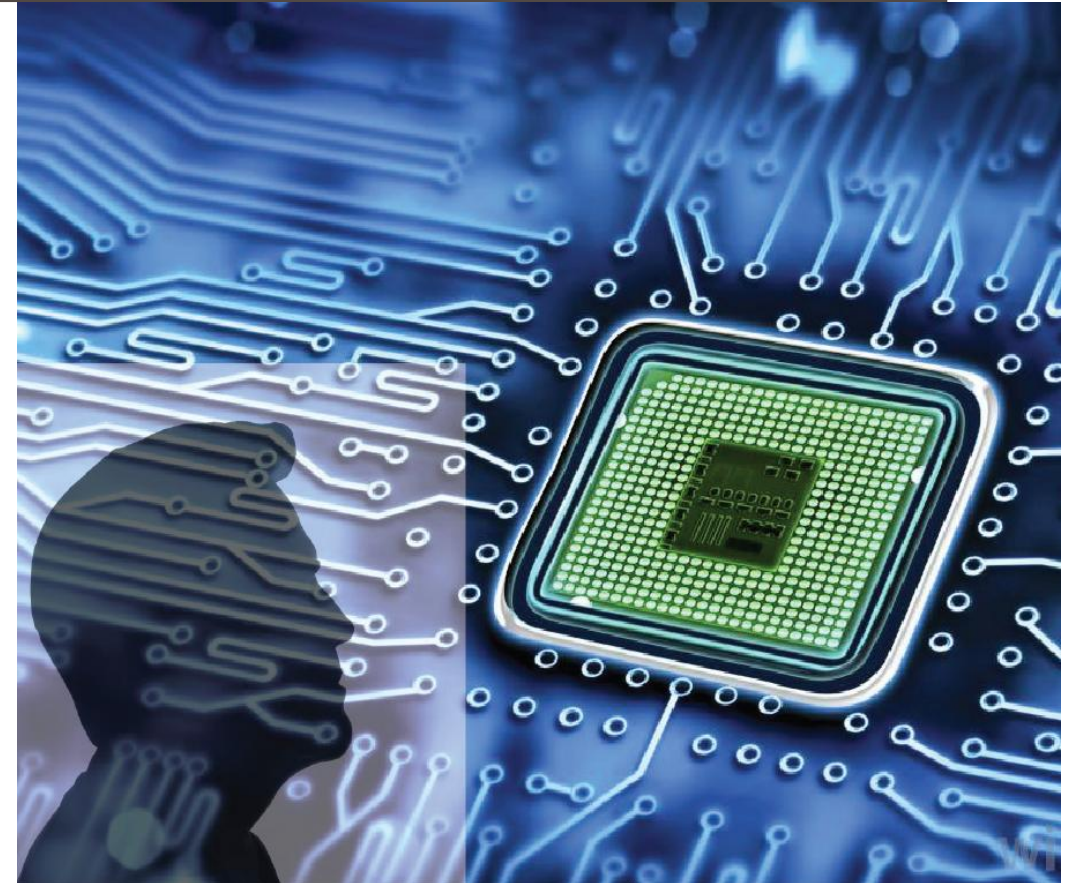


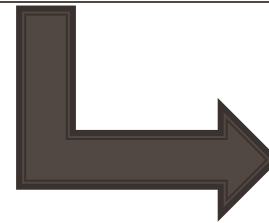
ADVANCED DIGITAL DESIGN AND VERIFICATION

Week-8 Lecture-2 SystemVerilog

Sneh Saurabh
9th October, 2025



SystemVerilog



Classes

Classes: Scoping Rules

Scope

- A block of code is called a scope
- Example: **module**, **program**, **task**, **function**, **class**, **for** block, **foreach** block, **begin-end** block

Defining scope of variables

1. Relative to the current scope
 - Looks up the list of scopes until it finds a match
2. Absolute, starting from **\$root**
 - **\$root** refers to global scope (outside any module, program or package)

- %m prints the hierarchical name

- To avoid ambiguity we can use **this** to specify class variable or object

```
program automatic top;

    int limit; // $root.top.limit

    class Foo;
        int limit, array[]; // $root.top.Foo.limit

        // $root.top.Foo.print.limit
        function void print (input int limit);
            for (int i=0; i<limit; i++)
                $display("%m: array[%0d]=%0d", i, array[i]);
            endfunction
        endclass

    initial begin
        int limit = 3; // $root.top.$unnamed.limit

        Foo bar;
        bar = new;
        bar.array = new[limit];
        bar.print (limit);
    end
endprogram
```

```
ncsim> run
top.Foo.print.unmblk1: array[0]=0
top.Foo.print.unmblk1: array[1]=0
top.Foo.print.unmblk1: array[2]=0
```

```
class Scoping;
    string name;

    function new(string name);
        this.name = name; // class name = local name
    endfunction

endclass
```

Classes: Containing Object of Other Class

```
1 typedef class Addr; // forward declaration needed
2
3 class Transaction;
4   Addr myaddr; // object of another class
5   function new();
6     myaddr = new();
7   endfunction
8   function void display;
9     $display("Transaction display");
10    myaddr.display();
11  endfunction : display
12 endclass
13
14 class Addr;
15   logic [31:0] addr;
16   function new();
17     addr = 32'hffff;
18   endfunction
19
20   function void display;
21     $display("Addr display: %h", addr);
22   endfunction : display
23 endclass
24
```

```
ncsim> run
Transaction display
Addr display: 0000ffff
Addr display: 0000ffff
```

```
25 module top;
26
27   initial
28   begin
29     Transaction t1;
30     t1 = new();
31     t1.display();
32     t1.myaddr.display();
33   end
34
35 endmodule
```

```
ncsim> run
Transaction display
Addr display: ncsim: *E,TRNULLID: NULL pointer dereference.
```

- For compilation, need to give a forward declaration
 - Must call **new()** for the contained handle to lower-level class
 - Variables and methods of the contained object can be referred to using dot (.)
 - If line number 6 is removed, it will crash
-
- When size of a class grows big, decomposing into smaller classes should be considered

Classes: Passing Object to Tasks/Methods (1)

- When function call is made with an object as an argument:
 - Handle to the object is passed and not the actual object
 - The caller can modify the object pointed to by the handle
 - Copy of the object is not created (as in the case of scalar)

```
19 module top;
20
21   initial
22   begin
23     Addr a;
24     a = new();
25     a.display();
26     corruptAddressScalar(a.addr); // scalar goes to function
27     a.display();
28     corruptAddressObject(a); // handle a goes to function
29     a.display();
30   end
31
32 endmodule
```

```
1 typedef class Addr; // forward declaration needed
2
3 task corruptAddressScalar(bit [31:0] address);
4   address[0] = ~address[0];
5 endtask
6
7 task corruptAddressObject(Addr addr);
8   addr.addr[0] = ~addr.addr[0];
9 endtask
10
11 class Addr;
12   bit [31:0] addr;
13
14   function void display;
15     $display("addr: %h", addr);
16   endfunction : display
17 endclass
18
```

```
ncsim> run
addr: 00000000
addr: 00000000
addr: 00000001
```

Classes: Passing Object to Methods (2)

- If we want to change handle itself, then we need to pass the reference to the handle

```
24 module top;
25
26   initial
27   begin
28     Addr a;
29     a = new();
30     a.flag = 129;
31     a.display();
32     corruptAddressObject(a); // handle goes
33     a.display();
34     corruptAddressNewObject(a); // reference to handle goes
35     a.display();
36   end
37
38 endmodule
```

```
ncsim> run
addr: 00000000 flag: 129
addr: 00000001 flag: 129
addr: 00000003 flag: 0
```

```
1 typedef class Addr; // forward declaration needed
2
3 class Addr;
4   bit [31:0] addr;
5   int flag;
6
7   function void display;
8     $display("addr: %h flag: %0d", addr, flag);
9   endfunction : display
10 endclass
11
12 task corruptAddressObject(Addr addr);
13   addr.addr[0] = ~addr.addr[0];
14 endtask
15
16 task automatic corruptAddressNewObject(ref Addr addr);
17   Addr newAddr;
18   newAddr = new();
19   newAddr.addr[0] = 1;
20   newAddr.addr[1] = 1;
21   addr = newAddr;
22 endtask
23
```

Classes: Copying Objects

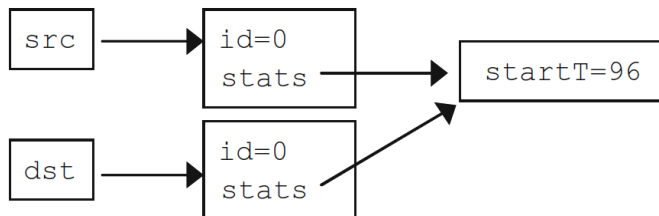
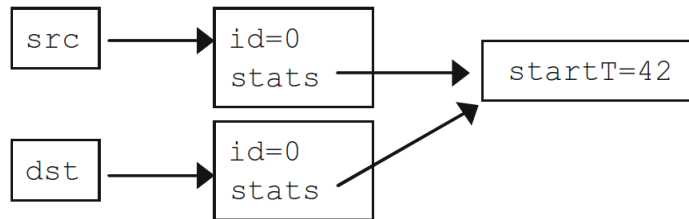
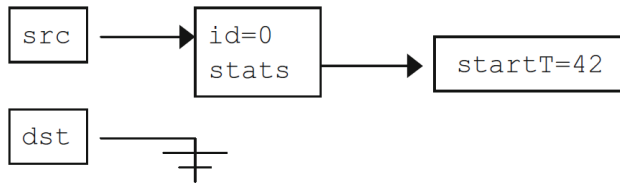
- Can use built-in copy available with the `new` operator
- Built-in copy allocates memory for the object
- All variables from the existing object are copied
- `new()` function defined for the class is not called

```
1 class Transaction;
2 bit data[8];
3 function new();
4     $display("In %m");
5 endfunction
6 endclass
7
8 module top;
9
10 Transaction src, dst;
11
12 initial begin
13     src = new(); // Create first object
14     dst = new src; // Make a copy with new
15 end
16 endmodule
```

In `$unit.Transaction.new`

Classes: Shallow Copy

- Built-in copy is a shallow copy
 - If a class contains a handle of an object of another class, only the handle is copied (not the entire object)



```
1 class Statistics;
2   time startT;
3 endclass
4
5 class BusTran;
6   static int count = 0;
7   int id;
8   Statistics stats;
9
10  function new;
11    stats = new;
12    id = count++;
13  endfunction
14 endclass
```

```
ncsim> run
Source: 42
Destination: 42
Source: 96
Destination: 96
```

```
16 module top;
17
18 BusTran src, dst;
19
20 initial begin
21   src = new; // Create first object
22   src.stats.startT = 42;
23   dst = new src; // Copy src to dst
24   $display("Source: %0d", src.stats.startT);
25   $display("Destination: %0d", dst.stats.startT);
26   dst.stats.startT = 96; // Changes stats for dst & src
27   $display("Source: %0d", src.stats.startT);
28   $display("Destination: %0d", dst.stats.startT);
29 end
30 endmodule
```


Classes: Defining Customized Copy

- Write `copy` function for a class
- Copy individual variable

```
1 class Transaction;
2 int data;
3
4 function new();
5     $display("In %m");
6 endfunction
7
8 function Transaction copy;
9     $display("In %m");
10    copy = new();
11    copy.data = data;
12 endfunction
13
14 endclass
```

```
17 module top;
18
19 Transaction src, dst;
20
21 initial begin
22     src = new(); // Create first object
23     $display("Src initial: %0d", src.data);
24     src.data = 104;
25     $display("Src modified: %0d", src.data);
26     dst = src.copy(); // Make a copy
27     $display("Dst initial: %0d", dst.data);
28 end
29 endmodule
```

```
ncsim> run
In $unit_0x3283e2f7.Transaction.new
Src initial: 0
Src modified: 104
In $unit_0x3283e2f7.Transaction.copy
In $unit_0x3283e2f7.Transaction.new
Dst initial: 104
```

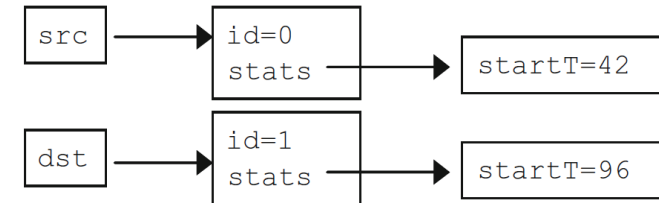
Classes: Defining Deep Copy

- Write copy function for a class
- Call copy function for all the contained objects

```
class Statistics;  
  time startT;  
  function Statistics copy();  
    copy = new();  
    copy.startT = startT;  
  endfunction  
endclass
```

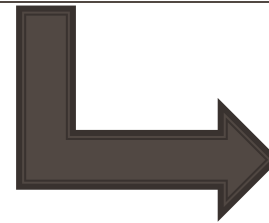
```
class BusTran;  
  bit [31:0] addr, crc, data[8];  
  Statistics stats;  
  static int count = 0;  
  int id;  
  
  function new();  
    stats = new();  
    id = count++;  
  endfunction  
  
  function BusTran copy();  
    copy = new(); // Construct destination  
    copy.addr = addr; // Fill in data values  
    copy.crc = crc;  
    copy.data = data; // Array copy  
    copy.stats = stats.copy(); // call Statistics:copy  
  endfunction  
endclass
```

```
module top;  
  
  BusTran src, dst;  
  
  initial begin  
    src = new(); // Create first object  
    src.stats.startT = 42;  
    dst = src.copy(); // Deep copy src to dst  
    dst.stats.startT = 96; // Changes stats for dst & src  
    $display(src.stats.startT);  
    $display(dst.stats.startT);  
  end  
endmodule
```



```
ncsim> run  
42  
96
```

SystemVerilog



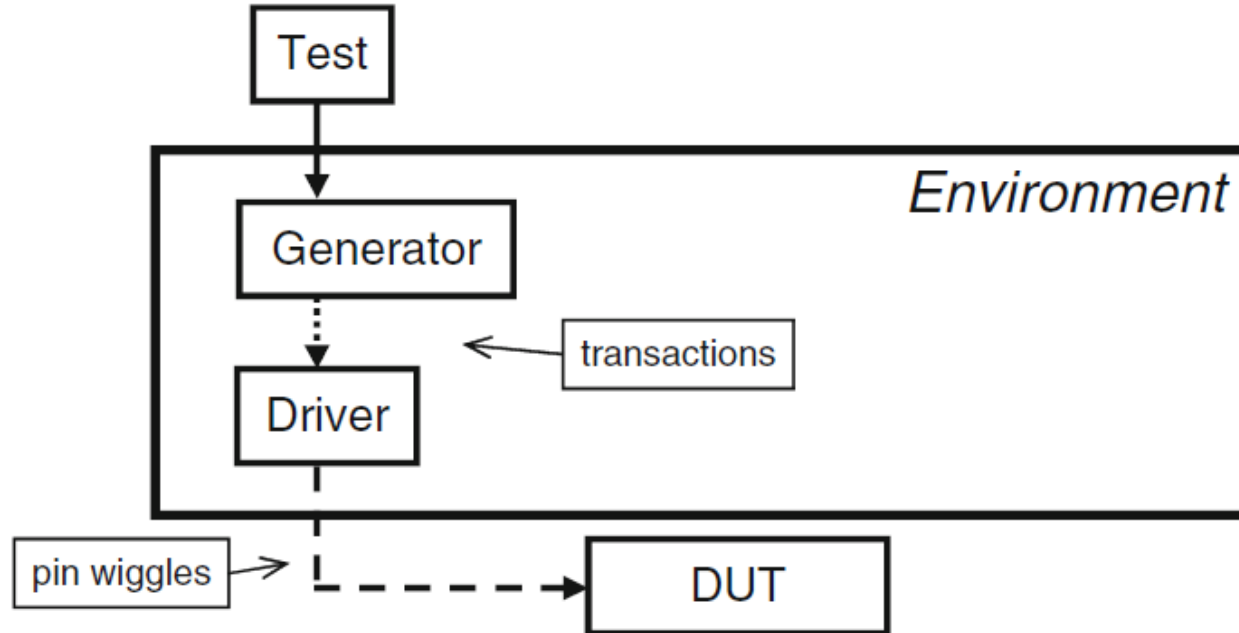
Inheritance

Inheritance

- Allows a new class to be extended from an existing one
 - By adding new variables and methods.
- The original class is known as the base class and the new class as the extended class

- Inheritance provides reusability
 - By overlaying features, such as error injection, on an existing class, without modifying that class
 - All your original code/tests that depend on the base class keep working
 - Can now create new features/tests with the extended class

Inheritance: Example

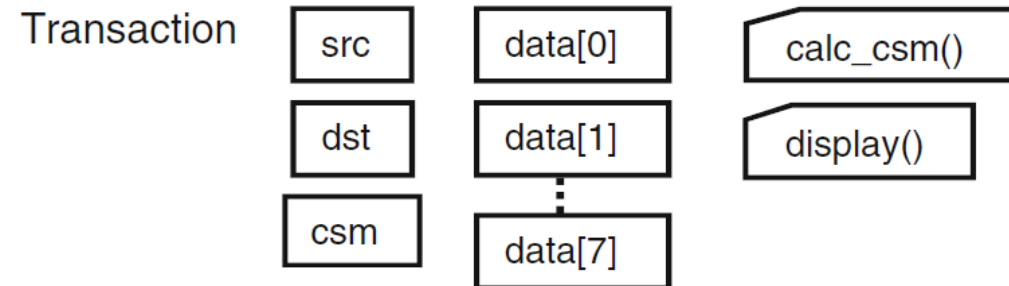


- Generator creates transactions, randomizes them, and sends them to the driver along the dotted line
- The driver breaks down the transaction into pin wiggles (signals on the pins) and sends it into the DUT along the dashed line.

- Transactions can be modeled as a base class and various other types of transactions can be extended from it

Inheritance: Base Class

```
class Transaction;  
  rand bit [31:0] src, dst, data[8]; // Variables  
  bit [31:0] crc;  
  
  virtual function void calc_crc();  
    crc = src ^ dst ^ data.xor;  
  endfunction  
  
  virtual function void display(input string prefix="");  
    $display("%sTr: src=%h, dst=%h, crc=%h",  
      prefix, src, dst, crc);  
  endfunction  
endclass
```



Inheritance: Extended Class

```
class BadTr extends Transaction;
  rand bit bad_crc;

  virtual function void calc_crc();
    super.calc_crc(); // Compute good CRC
    if (bad_crc) crc = ~crc; // Corrupt the CRC bits
  endfunction

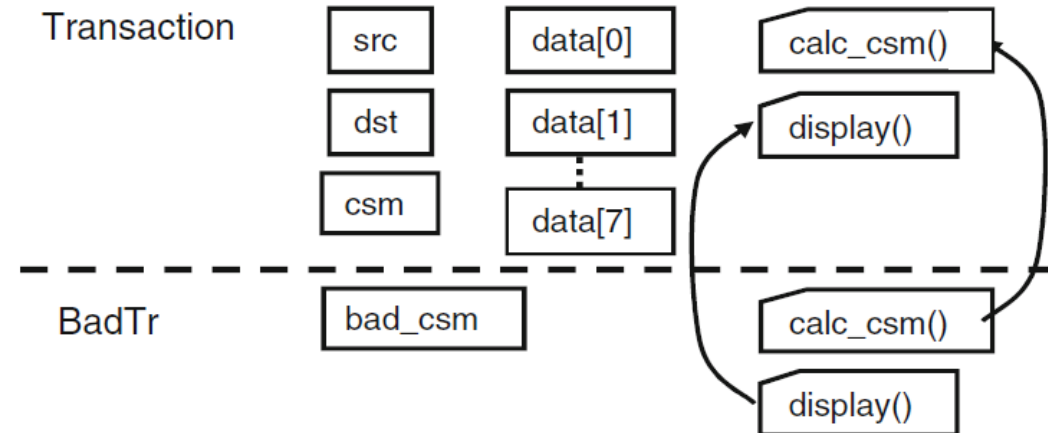
  virtual function void display(input string prefix="")
    $display("%sBadTr: bad_crc=%b, ", prefix, bad_crc);
    super.display();
  endfunction
endclass : BadTr
```

```
module top;
```

```
  Transaction tr;
  BadTr badt;
```

```
  initial begin
    tr = new; // Create first object
    tr.display("1.");
    badt = new;
    badt.display("2.");
  end
endmodule
```

- Can access data and methods of base class
- One level up using **super**
 - Multiple levels like super.super not allowed



```
ncsim> run
1.Tr: src=00000000, dst=00000000, crc=00000000
2.BadTr: bad_crc=0,
Tr: src=00000000, dst=00000000, crc=00000000
```


Inheritance: Constructor in Extended Class

- If the base class constructor has an argument, the extended class must:
 - have a constructor
 - In the above constructor it must call the base class's constructor on the first line

```
class Base1;  
  int var;  
  function new(int var); // Constructor with an argument  
    this.var = var;  
  endfunction  
endclass
```

```
class Extended extends Base1;  
  function new(int var);  
    super.new(var); // Must be first line of new  
    // Other constructor actions  
  endfunction  
endclass
```

Inheritance: Virtual Methods

- Allows calling the extended class function using the handle of base class
 - Function call is based on the object type and NOT the handle type

```
ncsim> run
1: Tr: src=00000000, dst=00000000, crc=00000000
2: BadTr: bad_crc=0
3: BadTr: bad_crc=0
```

- If there was no virtual keyword,
 - Function is called based on the handle type and NOT the object type

```
ncsim> run
1: Tr: src=00000000, dst=00000000, crc=00000000
2: BadTr: bad_crc=0
3: Tr: src=00000000, dst=00000000, crc=00000000
```

```
class BadTr extends Transaction;
    rand bit bad_crc;

    virtual function void display(input string prefix="");
        $display("%sBadTr: bad_crc=%b", prefix, bad_crc);
    endfunction
endclass : BadTr
```

```
class Transaction;
    rand bit [31:0] src, dst, data[8]; // Variables
    bit [31:0] crc;

    virtual function void display(input string prefix="");
        $display("%sTr: src=%h, dst=%h, crc=%h",
            prefix, src, dst, crc);
    endfunction
endclass
```

```
module top;

    Transaction tr;
    BadTr bad;

    initial begin
        tr = new;
        tr.display("1: "); // Calls Transaction::display

        bad = new;
        bad.display("2: "); // Calls BadTr::display

        tr = bad; // Base handle points to ext obj
        tr.display("3: "); // Calls BadTr::display
    end
endmodule
```

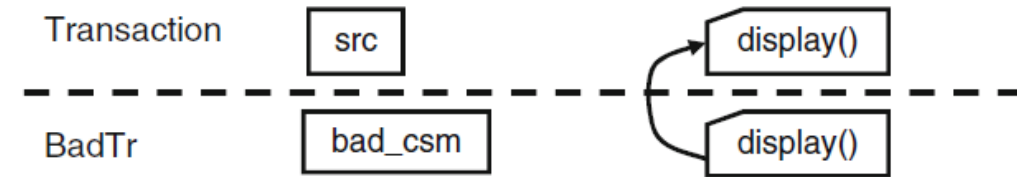
Inheritance: Casting

Down casting

- From Extended class handle to Base class handle
- No problem: extended class already has members of the Base class

Up casting

- From Base class handle to Extended class handle
- Not allowed: Base class may not have members of the Extended class



```
Transaction tr;  
  
BadTr bad;  
  
bad = new(); // Construct BadTr extended object  
tr = bad; // Base handle points to extended obj  
           // tr is downcast to point to BadTr (safe)  
$display(tr.src); // Display base variable  
tr.display; // Calls BadTr::display  
  
tr = new(); // Construct base object  
bad = tr; // ERROR: WILL NOT COMPILE  
$display(bad.bad_csm); // bad_crc is not in base object
```

Inheritance: \$cast

- Upcast allowed when the Base class handle actually points to an Extended class object
 - **\$cast** looks at the object type and not the handle type
- **\$cast** as task: checks for error during simulation
- **\$cast** as function: returns 0 if unsuccessful

```
class Transaction;  
    int data;  
  
endclass  
  
class BadTr extends Transaction;  
    bit wrong;  
endclass  
  
class TooBadTr;  
    bit wrong;  
endclass
```

```
14 module top;  
15     Transaction tr;  
16     BadTr bad;  
17     BadTr b2;  
18     TooBadTr tooBad;  
19  
20     initial begin  
21         bad = new;  
22         tr = bad; // Base handle points to extended object  
23         tooBad = new;  
24  
25         // Check object type and copy.  
26         // Simulation error for mismatch  
27         $cast(b2, tr);  
28  
29         if ($cast(b2, tr)) $display("First Successful");  
30         else $display("First Unsuccessful");  
31  
32         if ($cast(b2, tooBad)) $display("Second Successful");  
33         else $display("Second Unsuccessful");  
34     end  
35 endmodule
```

ncsim> run
First Successful
Second Unsuccessful

References

- Spear, Chris. SystemVerilog for Verification: A Guide to Learning the Testbench Language Features. Vol. 161. Springer, 2008.